Sorting Algorithms

Bubble Sort

What is Sorting?

- Sorting refers to ordering data in an increasing or decreasing fashion according to some linear relationship among the data items.
- Sorting can be done on names, numbers and records.

Some Methods Used for Sorting

- Insertion Sort
- Merge Sort
- Quick Sort
- Radix Sort
- Heap Sort
- Selection Sort
- ***Bubble Sort***

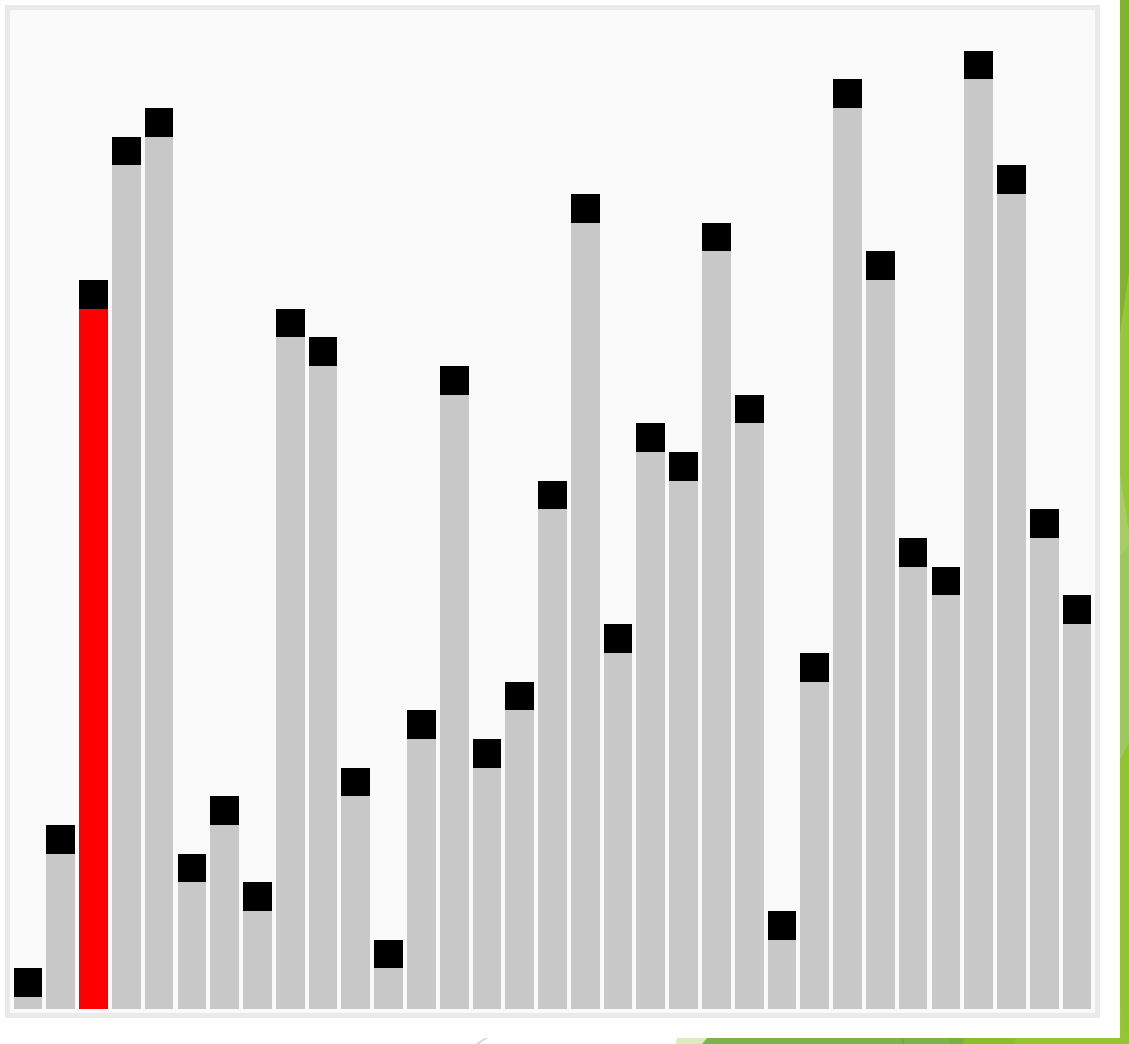
Bubble Sort

- One of the simplest methods
- Not suitable for large data sets
- Time consuming



Bubble Sort Algorithm

- Let’s take a look at the Bubble Sort algorithm
- This explains the ascending order



Requirements

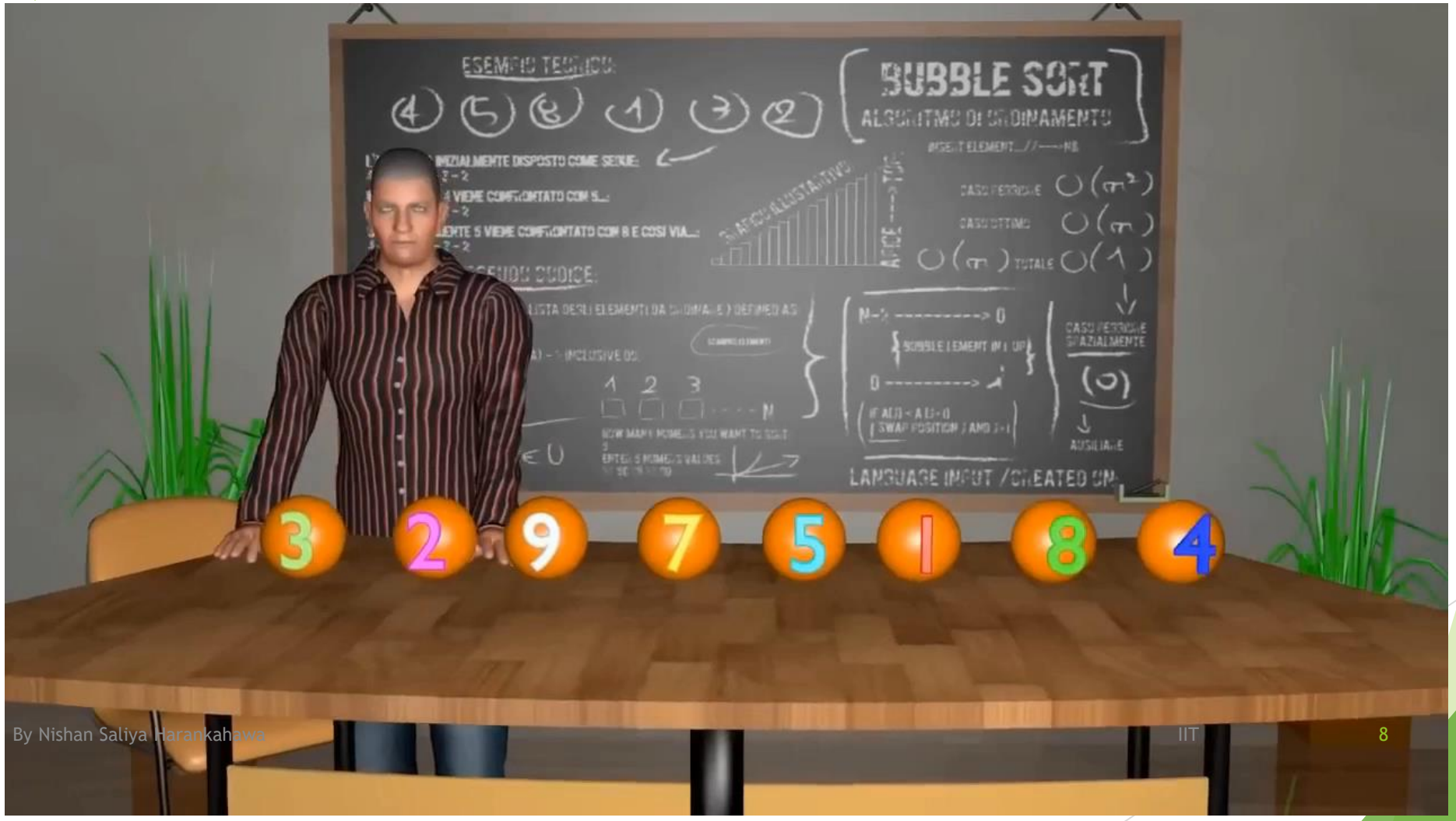
- Numbers must be in an array
  - For instance, a list
  - Covered in semester 1 and 2
- Loop through the list
  - Covered in semester 1 and 2
- Swap the two adjacent numbers
  - Covered in semester 1

Algorithm

1. Compare the first 2 elements
    1. Assume the values are 3 2 9 7 5 1 8 4
2. If the first element is higher than the second element, swap the two numbers. If not compare the second with third value.
3. Keep doing this till end of array (list)
4. Re-do the steps until the full array (list) is sorted

A Small Video Demonstration

(https://youtu.be/NiyEqLZmngY?t=52)



Version 1

```
# Declare variables and arrays
# NumberList = [1,2,3,4,5,7,8,9]
NumberList = [3,2,9,7,5,1,8,4]
temp = 0

# Display the original series
print('Before sort')
print(NumberList,end="\n\n")
print('During sort')

# Apply sorting
for i in range(7):
    if NumberList[i]>NumberList[i+1]:
        # Swaping occurs
        temp = NumberList[i]
        NumberList[i] = NumberList[i+1]
        NumberList[i+1] = temp
    print(NumberList)
```

Only one loop happens!

```
Before sort
[3, 2, 9, 7, 5, 1, 8, 4]

During sort
[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 7, 9, 5, 1, 8, 4]
[2, 3, 7, 5, 9, 1, 8, 4]
[2, 3, 7, 5, 1, 9, 8, 4]
[2, 3, 7, 5, 1, 8, 9, 4]
[2, 3, 7, 5, 1, 8, 4, 9]
```

Version 2

Two loops occur
Still sorted arrays are processed

```
# Declare variables and arrays
#NumberList = [1,2,3,4,5,7,8,9]
NumberList = [3,2,9,7,5,1,8,4]
temp = 0
ListSize = 8

# Display the original series
print('Before sort')
print(NumberList,end="\n\n")
print('During sort')

# Apply sorting
for i in range(ListSize-1):
    print()
    for x in range(0,ListSize-1):
        if NumberList[x]>NumberList[x+1]:
            # Swaping occurs
            temp = NumberList[x]
            NumberList[x] = NumberList[x+1]
            NumberList[x+1] = temp
        print(NumberList)
```

```
Before sort
[3, 2, 9, 7, 5, 1, 8, 4]

During sort

[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 7, 9, 5, 1, 8, 4]
[2, 3, 7, 5, 9, 1, 8, 4]
[2, 3, 7, 5, 1, 9, 8, 4]
[2, 3, 7, 5, 1, 8, 9, 4]
[2, 3, 7, 5, 1, 8, 4, 9]

[2, 3, 7, 5, 1, 8, 4, 9]
[2, 3, 7, 5, 1, 8, 4, 9]
[2, 3, 5, 7, 1, 8, 4, 9]
```

```
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
```

Things to Note

- The largest item will be moved to the last element when the loop run once
- Waste noted
  - Number of iterations are same even after sorted (You can improve this)
  - Sorting happens even if the list is sorted

Version 3

Two loops occur
Still sorted arrays are processed
Looping is reduced

```
# Declare variables and arrays
#NumberList = [1,2,3,4,5,7,8,9]
NumberList = [3,2,9,7,5,1,8,4]

temp = 0
ListSize = 8

# Display the original series
print('Before sort')
print(NumberList,end="\n\n")
print('During sort')

# Apply sorting
for i in range(ListSize-1):
    print()
    for x in range(0,ListSize-1-i):
        if NumberList[x]>NumberList[x+1]:
            # Swaping occurs
            temp = NumberList[x]
            NumberList[x] = NumberList[x+1]
            NumberList[x+1] = temp
        print(NumberList)
```

```
Before sort
[3, 2, 9, 7, 5, 1, 8, 4]

During sort

[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 9, 7, 5, 1, 8, 4]
[2, 3, 7, 9, 5, 1, 8, 4]
[2, 3, 7, 5, 9, 1, 8, 4]
[2, 3, 7, 5, 1, 9, 8, 4]
[2, 3, 7, 5, 1, 8, 9, 4]
[2, 3, 7, 5, 1, 8, 4, 9]
```

```
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]

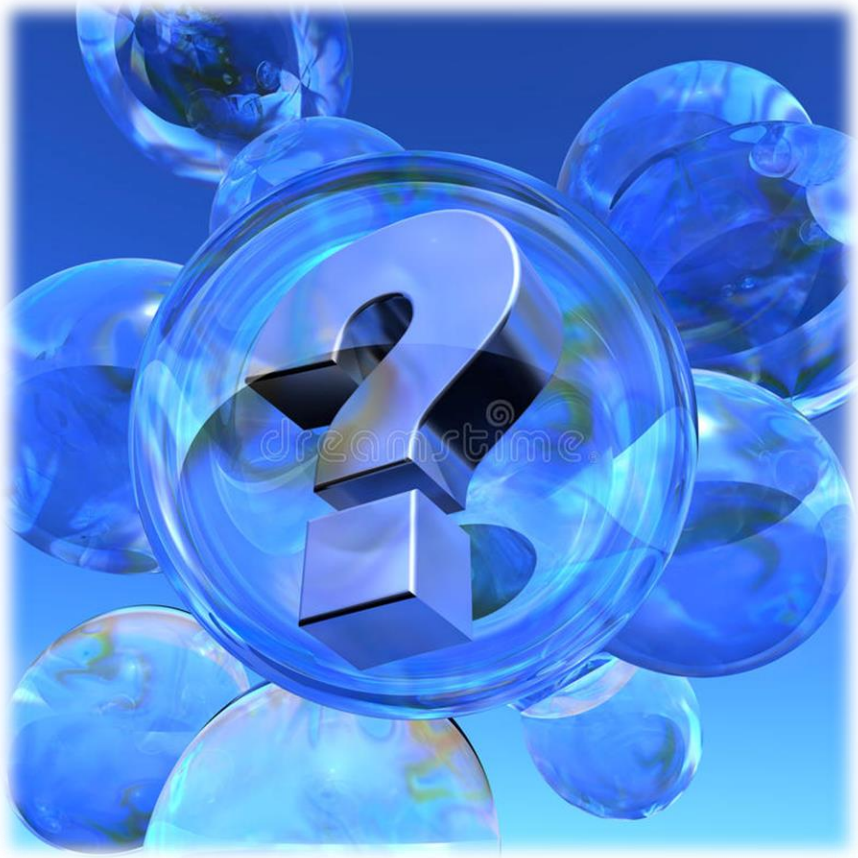
[1, 2, 3, 4, 5, 7, 8, 9]
[1, 2, 3, 4, 5, 7, 8, 9]

[1, 2, 3, 4, 5, 7, 8, 9]
```

Notes

- This is not the most optimized sort
  - Not required for L3 students
- You need to know the basic algorithm
  - Write down and explain the algorithm
  - Code it with Python
- Use proper Python options for a real program
  - Functions to carry out each task

Any Questions?



Reference

- https://www.geeksforgeeks.org/sorting-algorithms/
- https://ecomputernotes.com/data-structures/searching-and-sorting/what-is-sorting-type-of-sorting
- https://youtu.be/NiyEqLZmngY?t=52